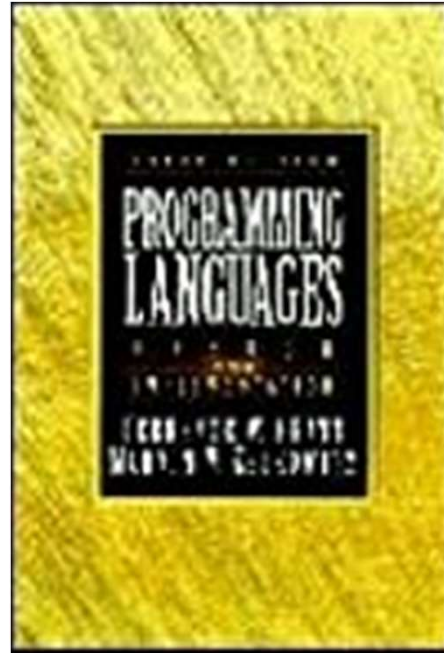




Controle de Subprograma

Interação entre Subprogramas

Parte 3 de 3

Ambiente Comum Explícito

66

- Um bloco de dados construído para compartilhar OD é o método mais direto para compartilhar dados entre os subprogramas que o declaram.
- Os OD são visíveis dentro dos subprogramas e usados normalmente.
- São declarados de forma diversa em diferentes LP:
 - ▣ common block em Fortran.
 - ▣ package em Ada.
 - ▣ variáveis etiquetadas como extern em C.
 - ▣ classes em C++ e Java (embora não seja esse o objetivo principal de se ter classes)

Ambiente Comum Explícito

Especificação

67

- Ambiente comum parece com um ambiente local mas **não é parte de nenhum subprograma.**
- Pode conter definições de variáveis, constantes e tipos.
- **Não contém** definição de **subprogramas e parâmetros:**
package Tab-partilhada is
 Tam-tab: constant integer := 100;
 type Tabela is array (1..Tam-tab) of real;
 Tab1, Tab2: Tabela;
 K: integer range 1..Tam-tab;
end;
□ O package (Ada) é **definido fora de qualquer subprograma** que usa as variáveis nele definidas.

Ambiente Comum Explícito

Especificação

68

- **Se P** é um subprograma que **usa Tab-partilhada** então P **precisa declarar “with Tab-partilhada”**:
procedure P is
with Tab-partilhada;
Tab-partilhada.Tab1(Tab-partilhada.K):=
 Tab-partilhada.Tab2(Tab-partilhada.K) + 10.5;
end;
- P pode usar os **nomes no pacote** como se fossem do seu ambiente local, mas as **referências são qualificadas** (nome do pacote é usado como prefixo).

Ambiente Comum Explícito

Especificação

69

- P pode compartilhar diversos pacotes. **Se não existem nomes locais em conflito**, então pode-se dispensar a qualificação, declarando após o “with” o comando **use**:
`with Tab-partilha;`
`use Tab-partilha;`
- Qualquer número de subprogramas pode **compartilhar o mesmo ambiente comum** e um subprograma pode **acessar qualquer número de ambientes comuns**.

Ambiente Comum Explícito

Implementação

70

- Em Fortran e C cada subprograma que usar ambiente comum **deve incluir declarações** para as variáveis que serão **compartilhadas**.
- Em Ada, o compilador recupera a declaração para o ambiente comum, ao encontrar o comando **with**, a partir de biblioteca ou de texto do programa.
- Em ambas, o compilador **inclui as declarações em uma tabela de símbolos**, como um conjunto adicional de nomes locais que podem ser referenciados no subprograma.
 - ▣ Isto permite **checagem estática de tipos**.

Ambiente Comum Explícito

Implementação

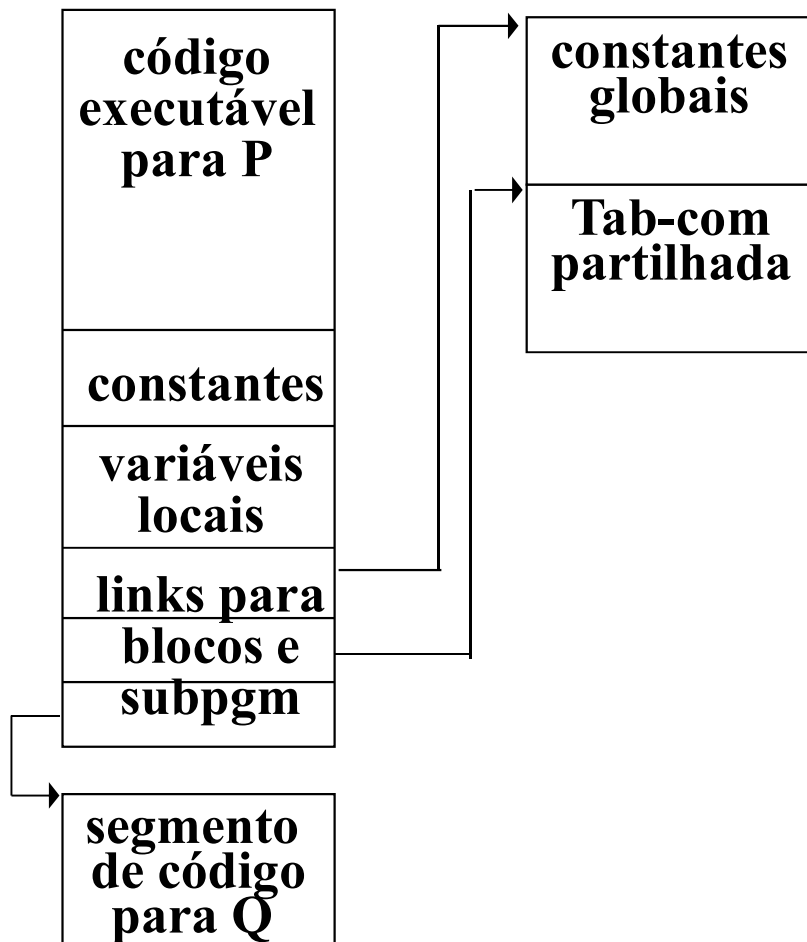
71

- Em tempo de execução:
 - ▣ O ambiente comum é representado como um **bloco contendo os OD declarados na sua definição**.
 - ▣ O bloco é **tratado** como um **registro** (base+offset).
 - ▣ O **nome** do bloco é o **nome do registro** e os seus **componentes** são as **variáveis declaradas**.
 - ▣ O bloco é **alocado estaticamente** no início da execução do programa e permanece até o final do mesmo.
 - ▣ O subprograma que **referencia um OD** no bloco deve **conhecer o endereço de base do bloco**.

Ambiente Comum Explícito

Implementação

72



- Uma implementação simples é colocar um **ponteiro no segmento de código** do subprograma P, apontando **para cada um dos blocos comuns** que utiliza.
- São utilizados **ponteiros** similares para indicar o **endereço de base dos subprogramas** chamados em P.
- A **inicialização** desses ponteiros é tarefa do **carregador de ligação** em tempo de carga (link editor).
- O **acesso** a cada variável de um bloco comum é dado por **base (r-value do ponteiro) + deslocamento**.

Ambiente Comum Explícito

Variáveis Compartilhadas Explicitamente

73

- Provê um meio de tornar um OD, no **ambiente local de um subprograma**, visível **para outros subprogramas**.
- Cada variável a ser compartilhada **possui um proprietário** (o subprograma que a declara).
 - ▣ O proprietário a compartilha com outros, **exportando-a** via uma declaração explícita como “**define**” em Ada ou **declarando-a fora de qualquer função** em C e C++
 - ▣ O subprograma que a deseja **importar** explicita intenção declarando “**uses**” em Ada ou “**extern**” em C ou C++.

Ambiente Comum Explícito

Variáveis Compartilhadas Explicitamente

74

□ Exportador (em Ada)

procedure **P** (...);

defines **X,Y,Z**;

X,Y,Z: real;

U,V: integer;

begin ... end;

- X, Y e Z (disponíveis para exportação).

- Declaração usual para X, Y e Z.

- Declaração de outras variáveis.

- Comandos.

□ Importador

procedure Q (...);

uses **P.X,P.Z**;

Y: integer;

begin ... end;

- uses <pgm>.<nome-variável-externa>

- Y é local, apenas X e Z são importadas.

- Comando pode incluir referência a X e Z

Ambiente Comum Explícito

Variáveis Compartilhadas Explicitamente

75

- Implementação
 - ▣ O efeito é similar ao uso de variável em um ambiente comum.
 - ▣ A variável é implementada como **retida**.
 - ▣ Variáveis **exportadas** são **armazenadas no segmento de código** de seu proprietário.
 - ▣ O **importador** usa o **endereço de base do proprietário** e o **deslocamento** apropriado para poder ter acesso a variável compartilhada.

Ambiente não Local Implícito

Escopo Dinâmico

76

- **Alternativa** a transmissão de parâmetros e a ambiente comum explícito para compartilhar dados é associação de **ambiente não local** a subprograma em **execução**.
 - ▣ O ambiente não local associado a P é o **conjunto de todos os outros ambientes locais de subprogramas que P enxerga**, durante sua execução.
 - ▣ Quando uma variável **X é referenciada em P** e **X não tem associação local** então o **ambiente não local é utilizado** para determinar a associação para X.

Qual deve ser o ambiente não local a P?

Ambiente não Local Implícito

Escopo Dinâmico

77

- Qual deve ser o ambiente não local a P?
 - ▣ Em **LP estruturadas em blocos** as **regras de escopo estático determinam o ambiente** não local explícito a cada subprograma.
 - ▣ Alternativa mais simples é usar os ambientes locais da **atual cadeia dinâmica de ativação** de subprogramas, **mesmo que eles não tenham definição aninhada** (LISP, APL e Snobol).
 - Essa ideia usa a **regra da associação mais recente**.

Ambiente não Local Implícito

Escopo Dinâmico

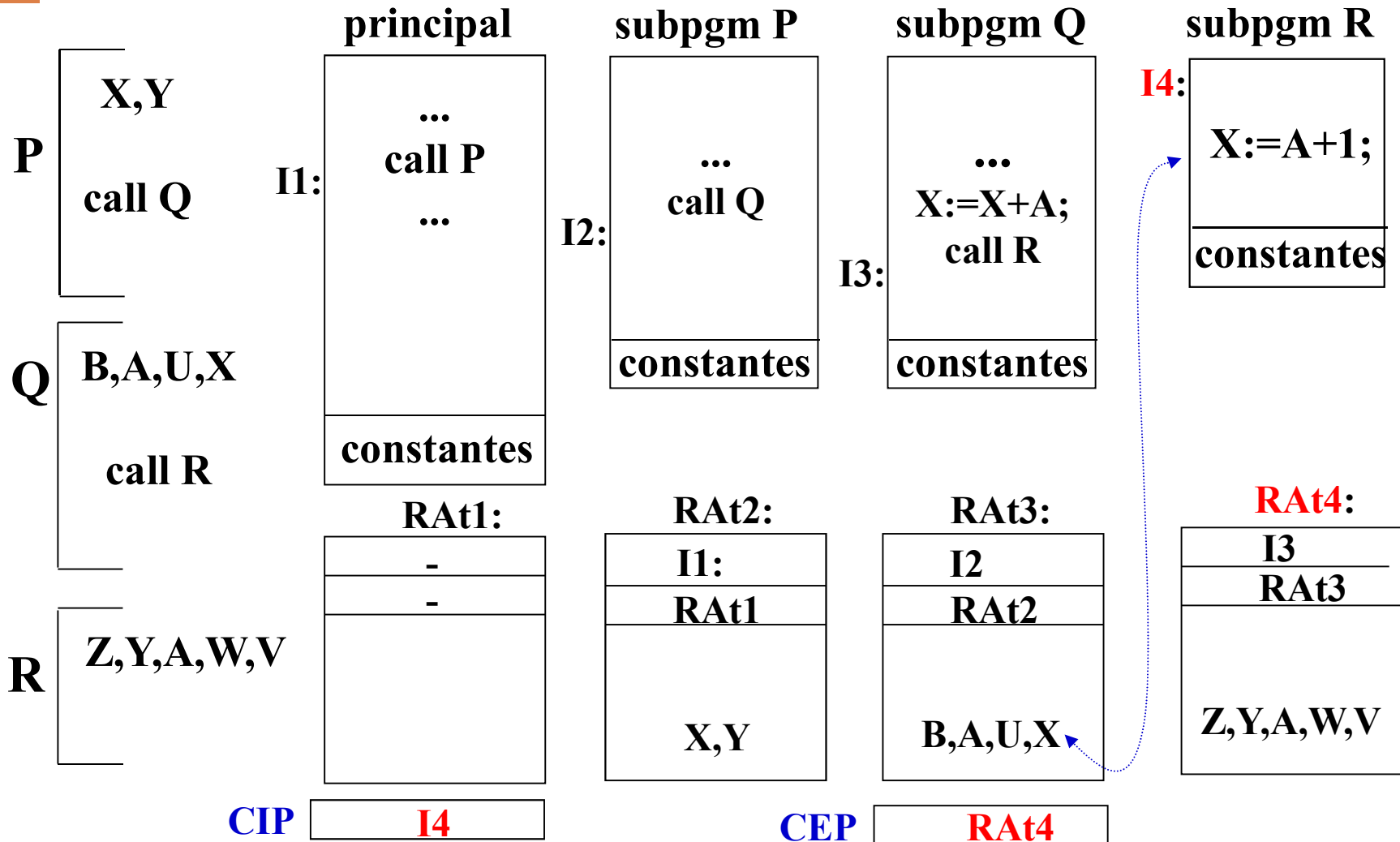
78

- **Regra da Associação mais recente:**
 - ▣ Aplicada quando a **LP não é estruturada em blocos** e:
 - os subprogramas são **definidos em separado** um dos outros.
 - os **ambientes locais são eliminados** ao término do subprograma.
 - **não existe estrutura de programa estática** na qual basear regras de escopo para referenciar identificadores não locais.
 - a LP utiliza **checagem de tipos dinâmica**.
 - ▣ Se **P referencia X** e **não existe uma associação para X em P**, então é **utilizada a associação mais recente** criada para X, na **cadeia dinâmica de ativações**.
 - ▣ **Ambiente não local** da ativação de P consiste da **cadeia dinâmica das ativações** de seus subprogramas **chamadores**.

Escopo Dinâmico

Regra da Associação mais recente

79



Os subprogramas são definidos em separados e não possuem estrutura em blocos

Ambiente não Local Implícito

Escopo Dinâmico

80

- Se a regra da associação mais recente **for aplicada**:
 - a **regra de escopo estático não se aplica** pois em tempo de tradução **não é feita** verificação de tipo ou associação de referências com declaração de variáveis.
 - o **ambiente não local pode variar entre ativações de P**, para atender a lógica de chamada dos subprogramas.
 - é necessário **armazenar uma tabela de símbolos locais** (isto é, identificadores) **em cada registro de ativação**.
 - a referência não local **não faz uso** de base mais deslocamento: **em cada registro de ativação** a base e o deslocamento **podem variar** para um mesmo identificador.

LP usa checagem de tipo dinâmica. Não tem declarações de variáveis. É interpretada.

Escopo Dinâmico

Regra da Associação mais recente

81

□ Implementação

a) Pilha Central (implementação direta)

- A pilha central é utilizada para **armazenar os registros de ativações (RAAt)** dos subprogramas.
 - O **ambiente local** de cada subprograma é parte do seu RAAt.
- Implementação cara: requer pesquisar o identificador nos RAAt
 - busca no **ambiente local** do ativo
 - **se não achar, segue a cadeia ativa em ordem reversa (caro)**
- O identificador é armazenado na tabela de símbolos local
 - **não pode** ser utilizado o esquema de **base-deslocamento**
 - a posição do identificador **pode variar** no RAAt onde aparece.

Pilha central é pouco utilizada. Mesmo identificador mas em posição diferente em cada subprograma

Escopo Dinâmico

Regra da Associação mais recente

82

□ Implementação

b) Tabela Central (evita busca por referência não local)

- É visível por todos os subprogramas, sendo chamada **Tabela de Ambiente de Referenciamento Central**:
 - contém **todas as associações ativas de identificadores** do programa, independente de serem ou não locais, **durante toda a execução**.
 - possui 3 campos: **identificador**, **flag** (indica se existe associação ativa para o identificador) e **valor** (ponteiro para o OD ligado).
- Se o **conjunto de identificadores** que pode ser referenciado em cada um dos subprogramas é **determinado durante a tradução**, a **tabela conterá uma única entrada para cada identificador**.

Vantajoso se o ambiente não local é mais usado do que modificado.

Regra da Associação mais recente

Implementação via Tabela Central

83

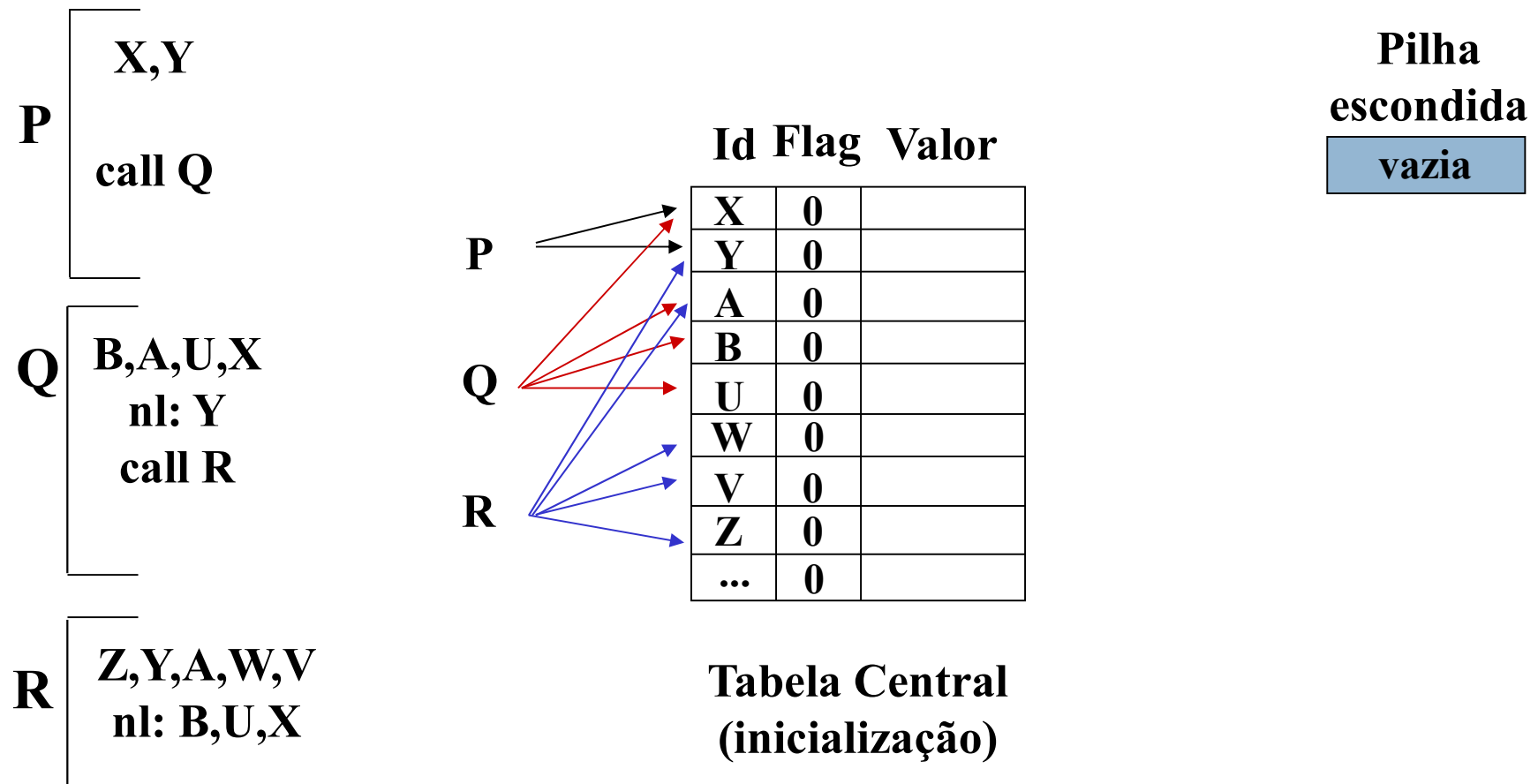
- Referências não locais são direcionadas para a tabela central: usa base mais deslocamento, evita uma pesquisa pelo identificador referido, se o **flag estiver ativo**.
- Na ativação do subprograma, os **identificadores ativos** (da tabela central) **que aparecem no ambiente local** são **salvos em uma pilha escondida** e **restaurados** quando o **subprograma termina**.
- Método é eficiente se a frequência de uso de variáveis não locais **justifica o maior custo de ativação e término dos subprogramas**, devido a manutenção da tabela central.

Regra da Associação mais recente

Implementação via Tabela Central

84

Tabela central ao término da tradução do programa fonte



Regra da Associação-mais-recente

Implementação via Tabela Central

85

Ativação de P, Q e R altera a tabela central e a pilha:

P	X,Y call Q	P ativo			Q ativo			R ativo			Q ativo		
		X	1	X1	X	1	X2	X	1	X2	X	1	X2
		Y	1	Y1	Y	1	Y1	Y	1	Y2	Y	1	Y1
		A	0		A	1	A1	A	1	A2	A	1	A1
		B	0		B	1	B1	B	1	B1	B	1	B1
		U	0		U	1	U1	U	1	U1	U	1	U1
		W	0		W	0		W	1	W1	W	0	W1
		V	0		V	0		V	1	V1	V	0	V1
		Z	0		Z	0		Z	1	Z1	Z	0	Z1
		...	0		...	0		...	0		...	0	
Q	B,A,U,X nl: Y call R												
R	Z,Y,A,W,V nl: B,U,X												

Pilha

vazia

X: X1

A: A1

Y: Y1

X: X1

X: X1

Ambiente não Local Implícito

Escopo Estático e Estrutura de Blocos

86

- Em **LP estruturadas em bloco** (Pascal, Ada e Algol), compartilhar dados via **referências não locais** é mais complexo e **não se pode usar regra de escopo dinâmico**
- O **ambiente não local** é determinado pelas **regras de escopo estático** (associando declarações e referências).
- A cadeia estática é determinada em tempo de tradução e assim uma **referência é resolvida** no ambiente **local** ou no **bloco** (subprograma) **mais próximo** que envolve o bloco onde a referência ocorreu.

Ambiente não Local Implícito

Escopo Estático e Estrutura de Blocos

87

- O significado de uma determinada referência não local a um identificador **não depende de uma particular cadeia dinâmica e ativação de blocos**, como no caso da regra da associação mais recente.
- Referência a um identificador em um subprograma é **associada com a sua declaração no fonte**, mesmo se o identificador for não local ao subprograma em questão

Ambiente não Local Implícito

Escopo Estático e Estrutura de Blocos

88

```
Main  x,y: integer;  
      [ y: real;  
        R  x:=x+1;  
          (* x é não local *)  
        Q  x: real;  
          R;  
        P  x: boolean;  
          Q;  
      begin  
        P;  
      end.
```

- O ambiente não local é determinado pelas regras de escopo estático: **x (em R) é inteiro e não real ou lógico.**
- **x em R** está associado a **x no programa principal** e independe de como R foi ativado (se através de P ou diretamente via Q).

Escopo Estático e Estrutura de Blocos

Implementação

89

Regras de Escopo Estático

- Na compilação de R , uma tabela de declarações locais (TL_R) é criada e **encadeada** em uma **lista de tabelas** que representa os ambientes locais do programa principal e de todos os subprogramas dentro dos quais R está aninhado.

$$TL_{main} \leftarrow TL_R$$

- Na compilação, o identificador é **buscado na TL_R** . Se não for encontrado, é pesquisado na **TL anterior** e **assim por diante**. x (em R) é associado ao x declarado no programa principal
- Quando a **compilação** de R **termina**, o compilador **elimina** TL_R da **lista de tabelas locais**.

$$TL_{main}$$

Utilizada checagem de tipo estática! Por isso as TL podem ser eliminadas ao término da compilação.

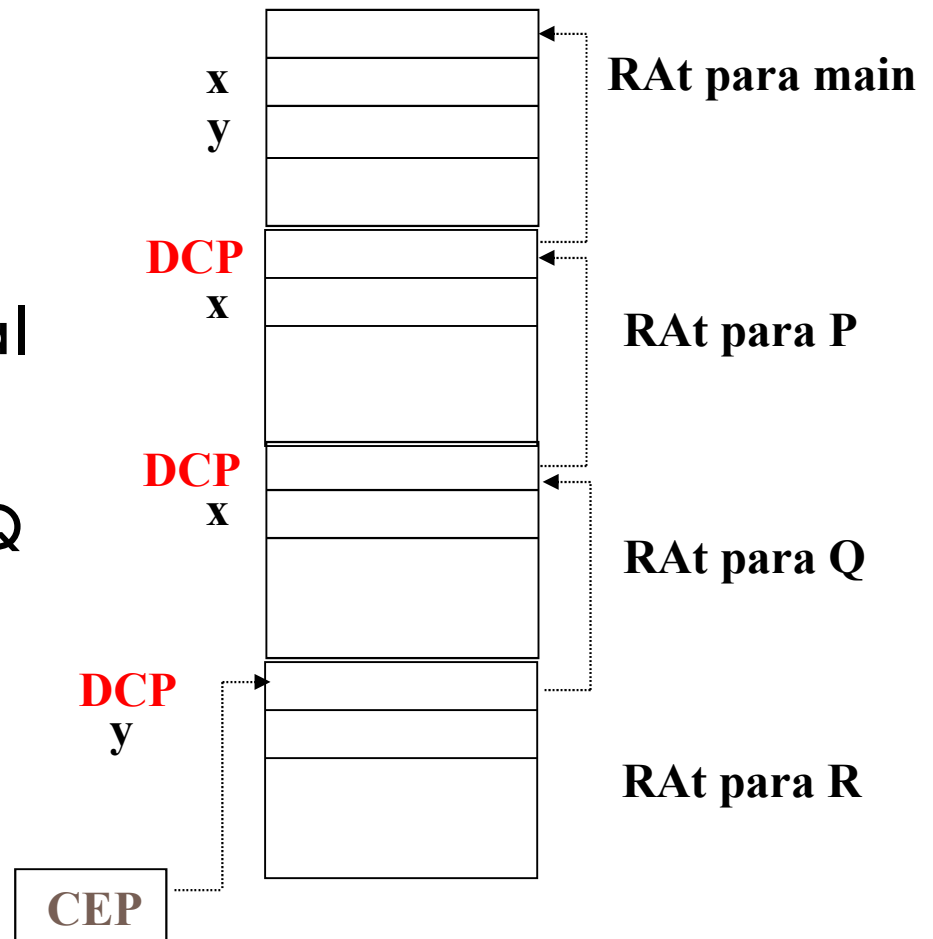
Escopo Estático e Estrutura de Blocos

Implementação

90

Pilha Central e Consistência entre os Escopos

- Em R a associação para x é com x no programa principal (**regra de escopo estático**)
 - **não** com o x declarado em Q (de acordo com a regra de escopo dinâmico).
- Como representar o escopo estático com **pilha central**?



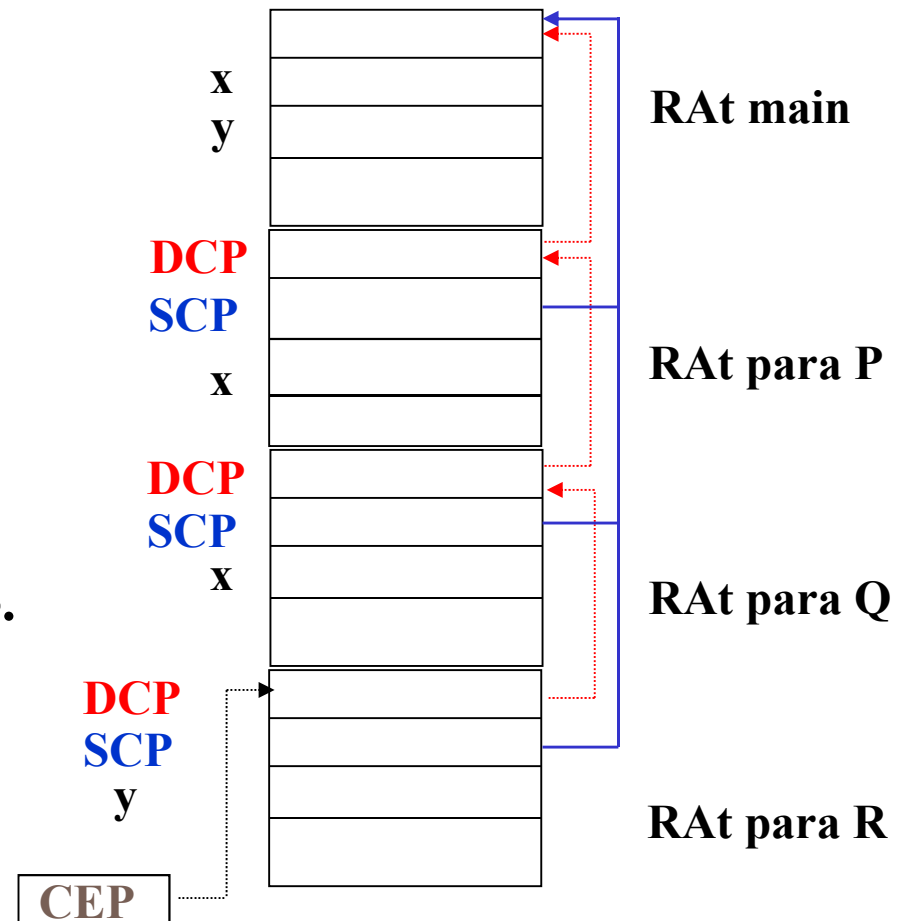
Pilha central representa as **chamadas dos subprogramas** durante a execução. DCP: dynamic chain pointer

Escopo Estático e Estrutura de Blocos

Implementação de cadeia estática

91

- Ambiente de referência não local é determinado pela **cadeia estática**, construída durante a **compilação**.
- ▣ Requer **ponteiro** para a TL do **bloco que precede** e que **envolve o bloco** em execução.
- ▣ O ponteiro de cadeia estática (SCP) aponta o **endereço de base do RAt do envolvente**.



Como um identificador é representado no código se a TL não é armazenada? SCP: static chain pointer

Escopo Estático e Estrutura de Blocos

Implementação de cadeia estática

92

- Compilador cria entradas na TL para declaração local
 - ▣ controla os subprogramas estaticamente aninhados.
 - ▣ Na referência a x o compilador **conta o número de ambientes envolvendo o subprograma em questão** e gera o código apropriado para seguir a cadeia estática até o RAt que contém o OD x adequado.
 - ▣ Referência a x é da forma **<número de passos atrás na cadeia estática, deslocamento no RAt que contém x >**
 - Identificadores não precisam ser armazenados no RAt ou segmento de código.
 - ▣ requer **n acessos** para acessar uma variável declarada em **bloco no nível k** e referenciada no nível **$k+n$** .

Escopo Estático e Estrutura de Blocos

Implementação de vetor mostrador

93

- Acessa OD não local com **uma referência** a um **ponteiro** da cadeia estática, sem percorrer
 - ▣ Para o subprograma, o **tamanho** da **cadeia** estática é **constante** (e igual ao nível do aninhamento).
 - Uma referência não local será satisfeita no **mesmo ponto na cadeia**, determinado em **tempo de tradução**.
- Um identificador é representado por (posição atrás, deslocamento no bloco)
 - ▣ **posição** (número de passos atrás) é um **ponteiro** na cadeia estática **para o RAt** onde o identificador é **definido**.

Também pode ser utilizado endereçamento indireto.

Escopo Estático e Estrutura de Blocos

Implementação de vetor mostrador

94

- No início do subprograma, a **cadeia estática** é **copiada** no **vetor display**
 - ▣ armazenado fora da pilha central, em **memória rápida**.
- Seja a referência da forma $x=(2,10)$,
 - ▣ o endereço (l-value) é dado por $\text{display}[2]+10$
 - deslocamento 10 no segundo RAt atrás, na cadeia estática
 - base + deslocamento.

Escopo Estático e Estrutura de Blocos

Declarações em Blocos Locais

95

- Algol e C permitem **declarações de variáveis** em blocos de comandos aninhados em procedimentos.

```
real proc1(...)
```

```
{ int i, j;
```

```
  ...  
  { int k, l;
```

```
    ...  
  }  
  { int m, n;
```

```
    ...  
    {int x; ...}  
    {int y; ...}
```

```
  }
```

```
}
```

A

B

C

D

- A e B são não simultâneos. C e D também são.
- Devido a não simultaneidade, essas declarações são implementadas via uma estrutura tipo registro de comprimento variável (**union** em C):

Outras variáveis		
i		
j		
k	m	
l	n	
	x	y